Exercises Week 7 Course 5 - Trees. Binary Trees

Exercises

1. Create the binary tree associated with a character sequence. The sequence is built with uppercase letters and special characters (,) $. The meaning of them is the following:
    a. Letters are the information in a node
    b. (is an inferior level
    c. , is a separation between the left and right subtree
    d.) marks the end of a level
    e. $ is the end of a subtree

How to create a binary tree? What does that mean?

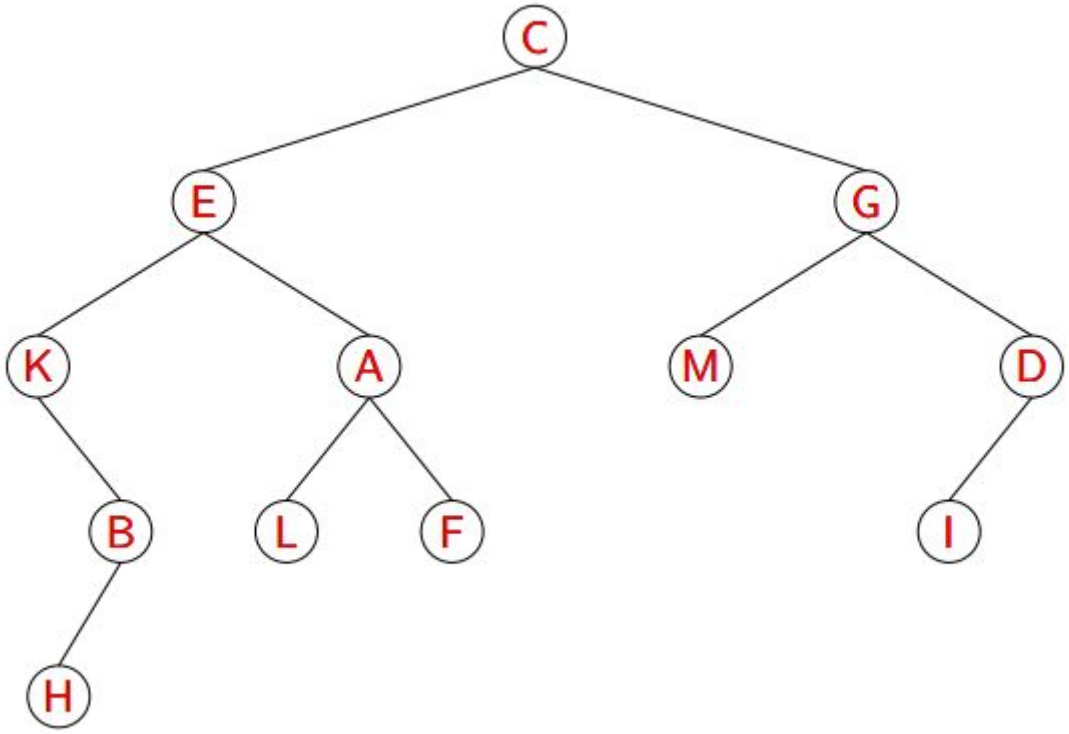
Exercises Week 7 Course 5 - Trees. Binary Trees

It means a “linked list” of the following structure:

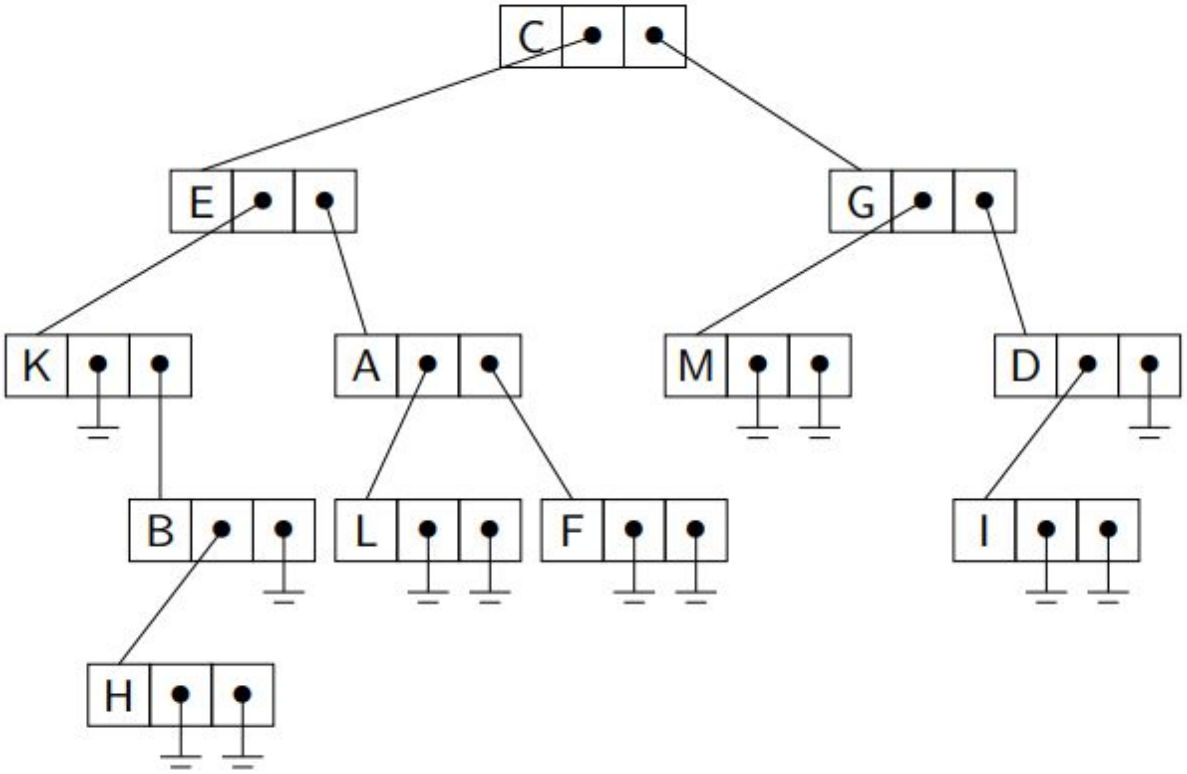
```
struct Node{
        Integer/Double… Value
        Node *Left
        Node *Right
}
```

Exercises Week 7 Course 5 - Trees. Binary Trees

Example:



Is the following structure:



Exercises Week 7 Course 5 - Trees. Binary Trees

So, we must, again.

Create the binary tree associated with a character sequence. The sequence is built with uppercase letters and special characters (,) $. The meaning of them is the following:

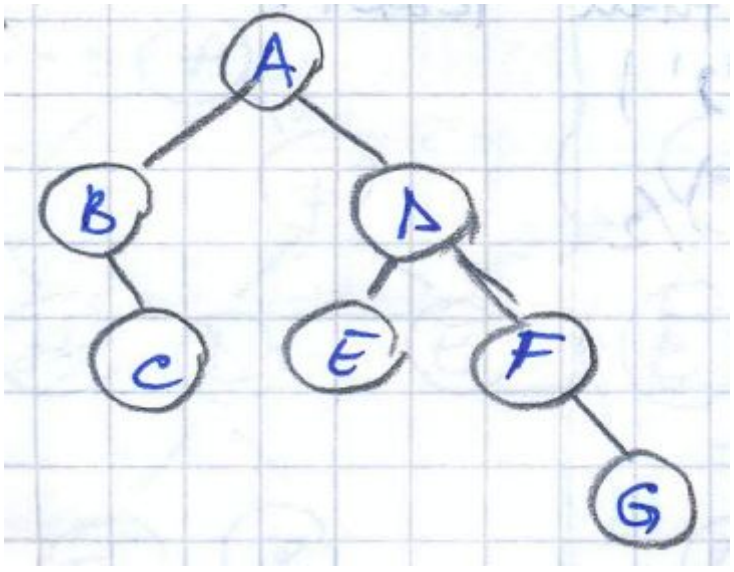
a. Letters are the information in a node
b. (is an inferior level
c. , is a separation between the left and right subtree
d.) marks the end of a level
e. $ is the end of a subtree

Input: A string of characters, sir[0,n-1] and an n representing the length of the string.
Output: A binary tree, which is represented with the structure node, described above, in the form of a linked list.

Example:

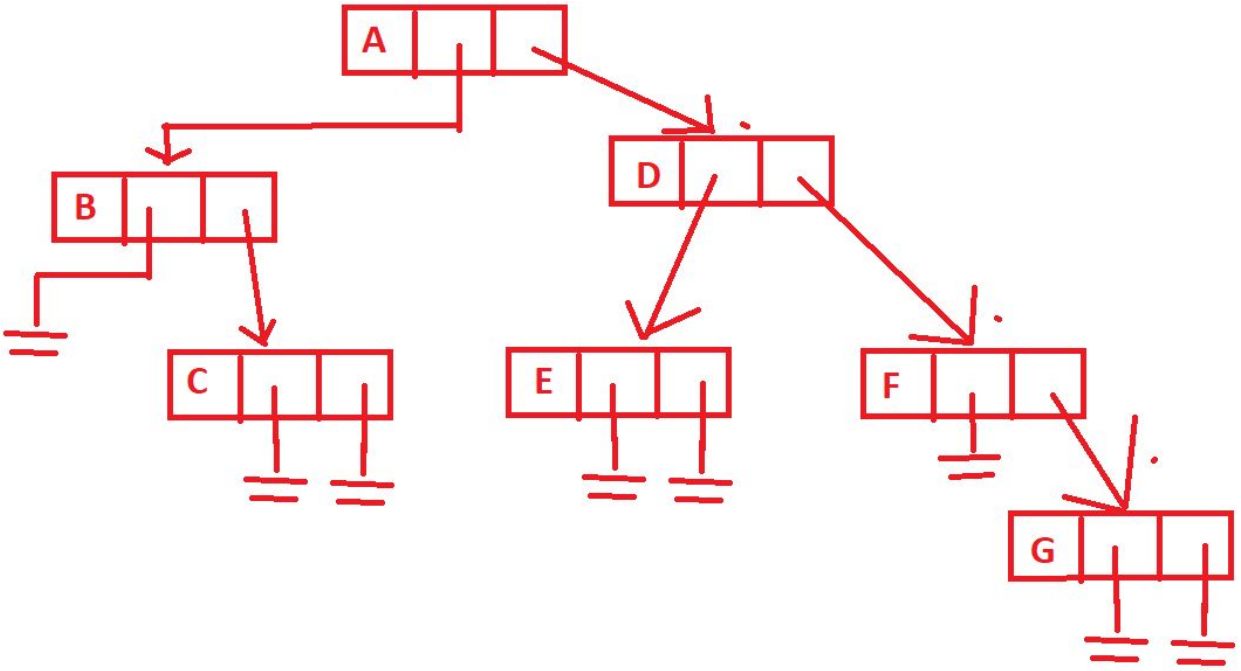
Input: “A(B($,C),D(E,F($,G)))” and the length n = 21

This is basically this tree:



Exercises Week 7 Course 5 - Trees. Binary Trees

Output: a linked list of type node, like this:



Exercises Week 7 Course 5 - Trees. Binary Trees

Suggestions?

Exercises Week 7 Course 5 - Trees. Binary Trees

Hint, we use a stack.

Exercises Week 7 Course 5 - Trees. Binary Trees

What would the rules of the stack be?

Exercises Week 7 Course 5 - Trees. Binary Trees

Every time we get a character we create a node

```
struct Node{
        string Value
        Node *Left
        Node *Right
}
```

With value = string[i]

Left is null
Right is null

Next what? What is $?

Exercises Week 7 Course 5 - Trees. Binary Trees

Every type we get to $ - this means NULL, we put null in the stack

Next what? What if we reach “)”?

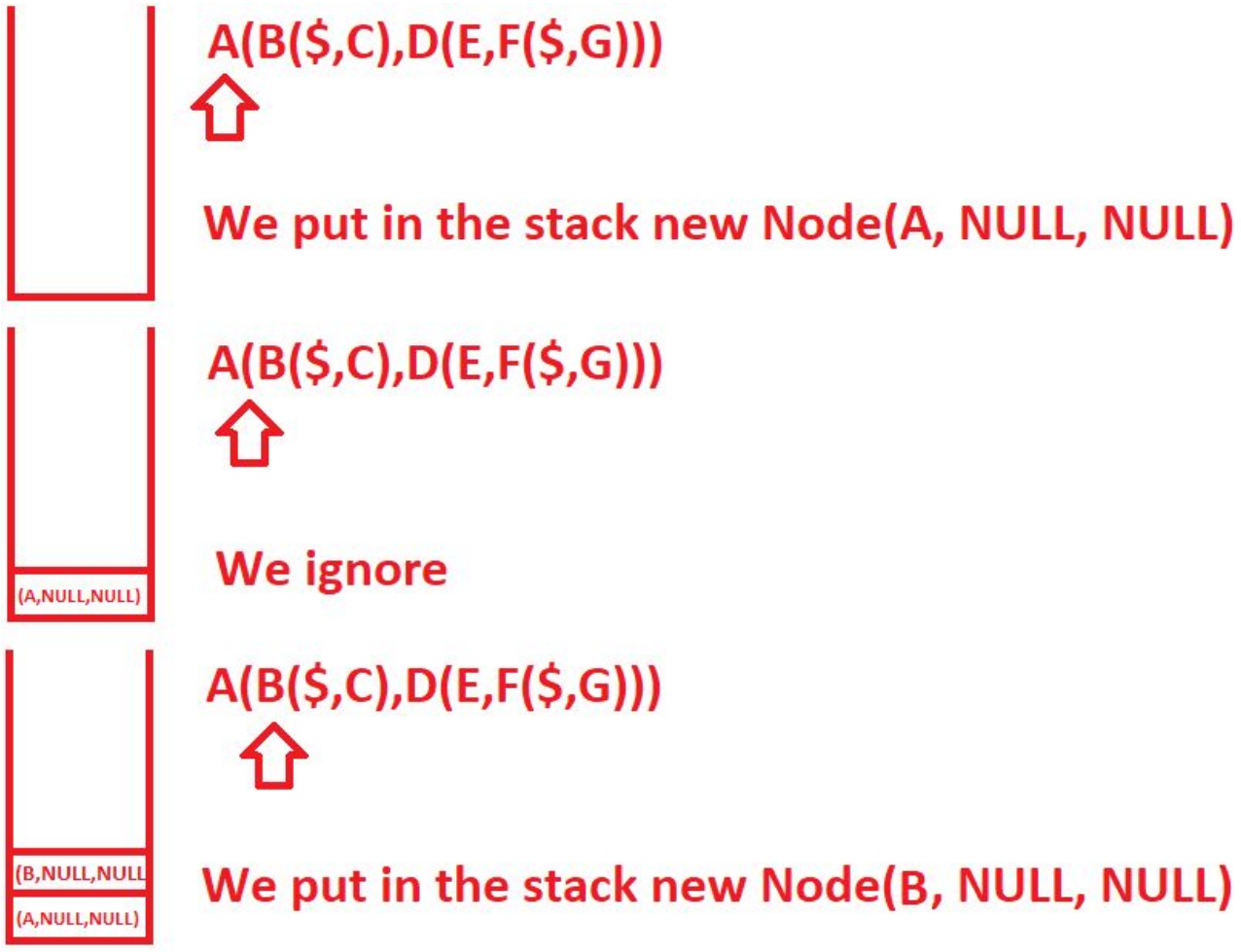
Exercises Week 7 Course 5 - Trees. Binary Trees

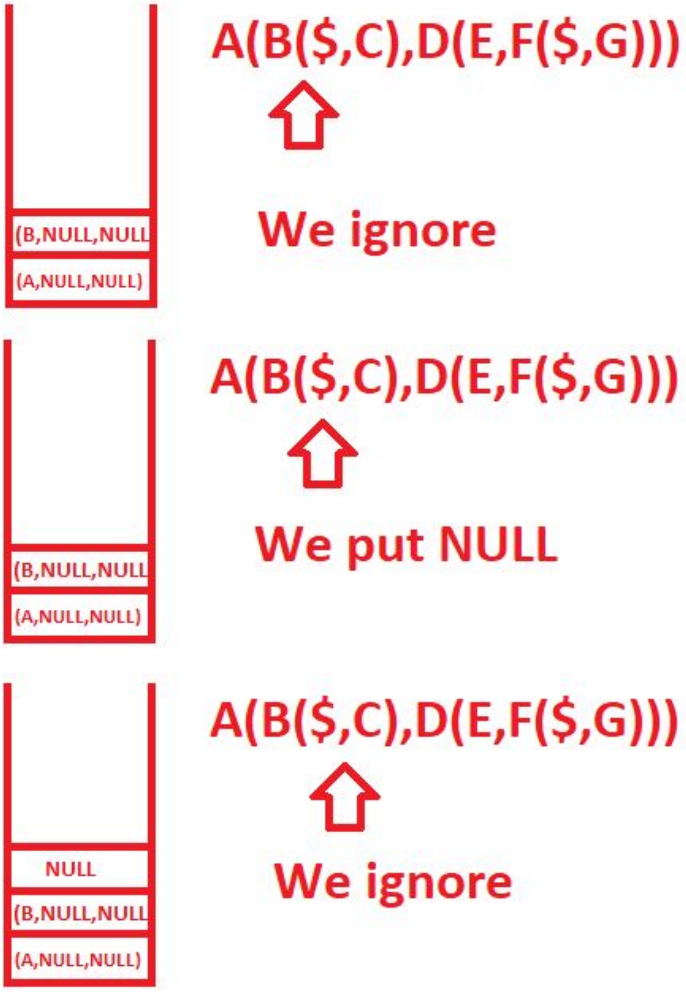
If we reach a “)” this means that we are ending a level?

Since we have a binary tree this means we have to pop 3 times. 2 times to get the children and once to get the parent.

Exercises Week 7 Course 5 - Trees. Binary Trees

Example:




A(B($,C),D(E,F($,G)))
We ignore
(B,NULL,NULL
(A,NULL,NULL)
A(B($,C),D(E,F($,G)))
We put NULL
(B,NULL,NULL
(A,NULL,NULL)
A(B($,C),D(E,F($,G)))
We ignore
NULL
(B,NULL,NULL
(A,NULL,NULL)

A(B($,C),D(E,F($,G)))

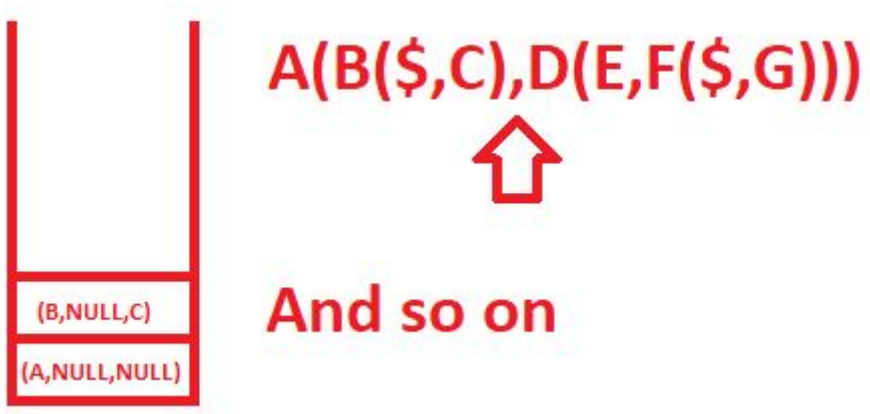
We put new Node(C,NULL,NULL)

A(B($,C),D(E,F($,G)))

Now is now, we reached a "closing level" character.

We must pop the stack 2 times to get the first 2 children.

Then pop once for the root, add to the left and to the right the popped elements then put it back in the stack and repeat


(B,NULL,C)
(A,NULL,NULL)
A(B($,C),D(E,F($,G)))
And so on

Exercises Week 7 Course 5 - Trees. Binary Trees

5 minutes to see if you can solve it yourself

Exercises Week 7 Course 5 - Trees. Binary Trees

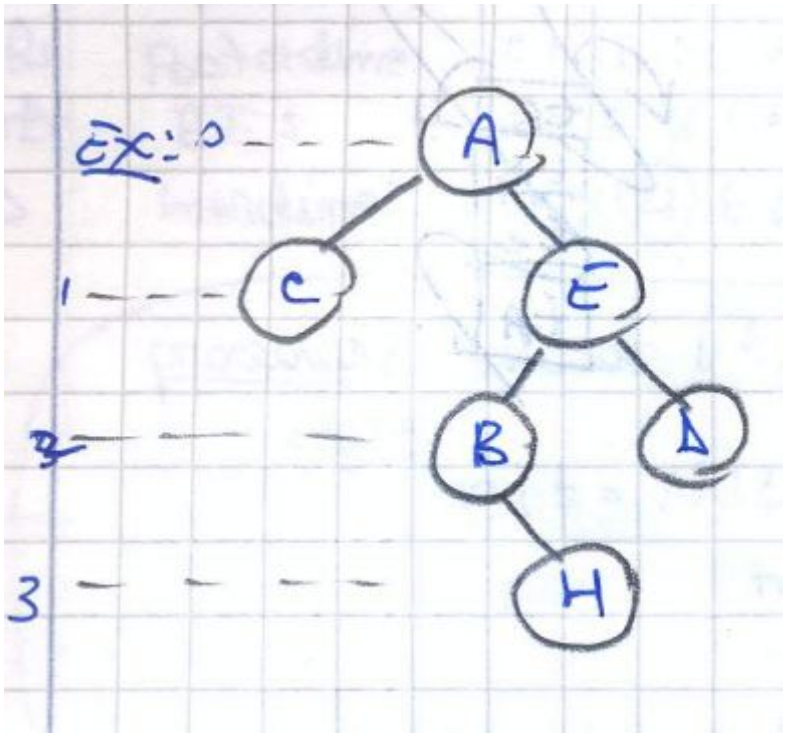
```
Function build_tree(string, n)
begin
        stack = emptyStack()
        For i = 0 to n-1 do:
                If isChar(string[i]) then
                        p = new node()
                        p -> value = string[i]
                        p -> st = NULL
                        p -> dr = NULL
                        push(stack, p)
                else If string[i] == “$” then
                                push(stack, NULL)
                else if string[i] == “)” then
                        a = top(stack)
                        pop(stack)
                        b = top(stack)
                        pop(stack)
                        root = top(stack)
                        pop(stack)

                        root -> st = b
                        root -> dr = a
                        push(stack, root)

        return top(stack)
end
```

Exercises Week 7 Course 5 - Trees. Binary Trees

We will discuss another solution for this problem.

Besides the representation as a linked list we can represent it as an **array**.

For example: “A(C,E(B($,H),D))” with the following tree



Exercises Week 7 Course 5 - Trees. Binary Trees

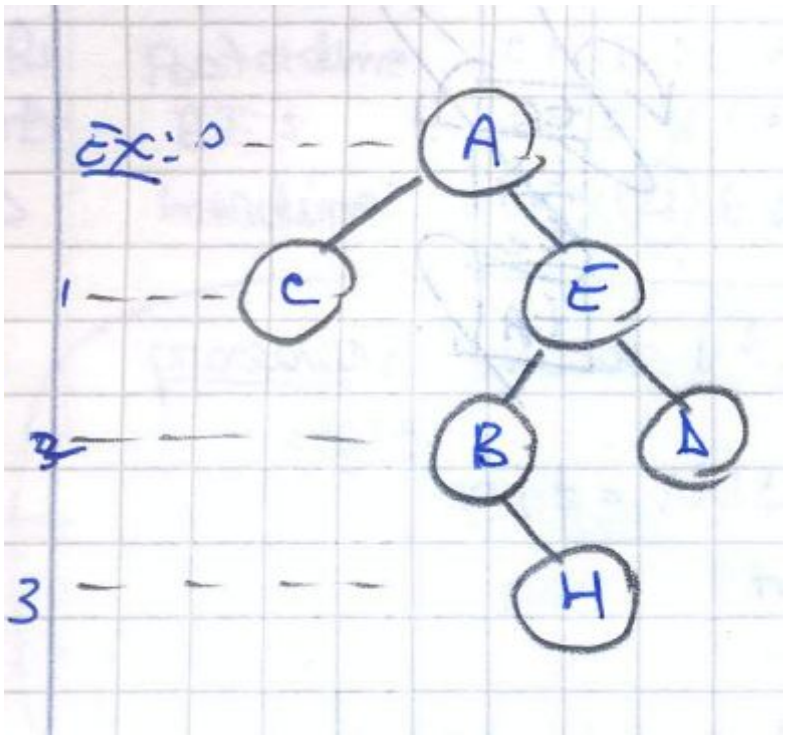
An array representation will have the following rules:

i = 0 - will be the root.

The rest of the nodes will follow the formula, for every **i** his children will be at the following position:

- 2*i+1 for the left
- 2*i+2 for the right

Example, for the tree below, for “A(C,E(B($,H),D))”



We will take every node in order and insert it in an array.

First we have A: (A,...)

Then we have C: (A,C,...)

Then we have E: (A,C,E,...)

Then we have NULL (C does not have children), (A,C,E,NULL,...)

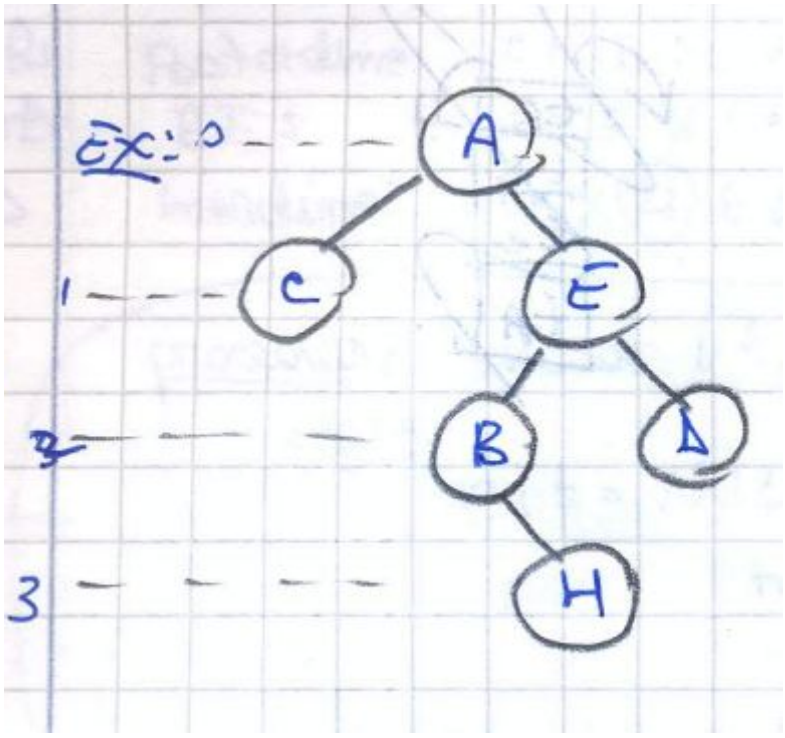
Then we have NULL (C does not have children), (A,C,E,NULL,NULL,...)

Then we have B, (A,C,E,NULL,NULL,B,...)

Then we have D, (A,C,E,NULL,NULL,B,D...)

Then, on level 3, until H we should have 4 leaves, (C should have 2 children, and they should have 2 children) hence we put 4 NULLs => (A,C,E,NULL,NULL,B,D,NULL,NULL,NULL,NULL...)

Exercises Week 7 Course 5 - Trees. Binary Trees



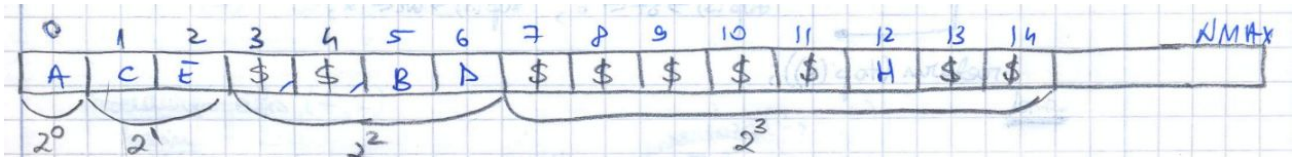
Then B has no children on left, hence NULL,
(A,C,E,NULL,NULL,B,D,NULL,NULL,NULL,NULL,NULL...)

Then B has a child on right, hence H, (A,C,E,NULL,NULL,B,D,NULL,NULL,NULL,NULL,NULL,H...)

Then D does not have 2 children, so 2 NULLs

(A,C,E,NULL,NULL,B,D,NULL,NULL,NULL,NULL,NULL,H,NULL,NULL...NULL)

The length of the vector will be 2^h-1 so we can account for all the graph possibilities, and we must first pad it with NULL.



If we are on 0 the root is A and has 2 children at 2*i+1 and 2*i+2 which are

- 2*0+1 = 1 -> C
- 2*0+2 = 2 -> E

Which is correct.

Exercises Week 7 Course 5 - Trees. Binary Trees

So, recap:

1. Init a vector with size 2^h-1 with $
2. Parse the string with the same rules as before (with minor changes)
    a. Instead of using a stack, use the vector and a position index k
    b. First k is 0
    c. If we are on a char we put the vector[k] to the string[i]
    d. If we are on an open position we must move the k with 2*k+1
    e. If we are on a , we move with 1
    f. If we close) we move k to (k-1)/2

5 minutes, go

You can use NMAX to signify 2^h-1

Exercises Week 7 Course 5 - Trees. Binary Trees

```
Function build_tree(string, n)
Begin
        tab[0,NMAX]
        k=0
        for i = 0 to NMAX do
                tab [i] = '$'
        for i = 0 to n-1 do
                if isChar(string[i]) then
                        tab[k]=string[i]
                Else if string[i] == '(' then
                        K = 2*k+1
                Else if string[i] == ',' then
                        k  = k +1
                Else if string[i] == ')' then
                        K = (k-1)/2
        return tab
End
```

Exercises Week 7 Course 5 - Trees. Binary Trees

2. For the following expression “A(B,C(D(E,F),$))” create the tree and tell:
*the preorder - preordine - RSD - traversal
*the inorder - inordine - SRD - traversal
*the postorder - postording - SDR - traversal

Exercises Week 7 Course 5 - Trees. Binary Trees

“A(B,C(D(E,F),$))”

*the preorder - preordine - RSD - traversal - A B C D E F
*the inorder - inordine - SRD - traversal - B A E D F C
*the postorder - postording - SDR - traversal - B E F D C A

Exercises Week 7 Course 5 - Trees. Binary Trees

3. BFS traversal of the previous tree “A(B,C(D(E,F),$))”

Exercises Week 7 Course 5 - Trees. Binary Trees

Will result in A B C D E F